

---

Tanta University — Faculty of Engineering  
**Computers & Network Security · CCE 4236 / CCE 442**

# Caesar & Vigenère Cipher

Implementation, Cryptanalysis, and Security Analysis

---

## UNIVERSITY

Tanta University — Faculty of Engineering

## COURSE

Computers and Network Security (CCE 4236 & CCE 442)

## LEVEL

Third and Fourth Year — Academic Year 2026–2027

## SUBMITTED TO

Dr. Tahani Allam

---

## PROJECT TEAM

**Ahmed Mohammad Fayad**

**Ahmed Alaa Bahnasy**

**Mostafa Moheb Abo Elmakarim**

**Ahmed Abd Elwhab ElWakil**

---

## ● SECTION 1

## Caesar Cipher

The Caesar cipher is one of the oldest known encryption techniques — simple enough to describe in a single sentence, yet historically significant as a foundation of classical cryptography. Every letter in the plaintext is shifted forward by a fixed number of positions in the alphabet, producing the ciphertext. Julius Caesar himself reportedly favoured a shift of 3 for military correspondence, trusting that most of his adversaries were illiterate. The cipher is symmetric: the same key that encrypts also decrypts, provided you subtract rather than add.

### Encryption & Decryption Formulae

**Encrypt:**  $C = (P + \text{key}) \bmod 26$

**Decrypt:**  $P = (C - \text{key} + 26) \bmod 26$

*P is the 0-based position of the plaintext letter (A=0 ... Z=25) and C is the resulting ciphertext position. The +26 in decryption prevents negative modulo results. Non-alphabetic characters — spaces, digits, punctuation — pass through unchanged, and letter case is preserved.*

### Step-by-Step Example — Encrypting "HELLO" with key 3

Each letter of "HELLO" is shifted forward by 3 positions. H (position 7) becomes K (position 10); E (4) becomes H (7); each L (11) becomes O (14); and O (14) becomes R (17). The full transformation reads:

| Letter | Position (P) | P + 3 | mod 26 | Encrypted |
|--------|--------------|-------|--------|-----------|
| H      | 7            | 10    | 10     | K         |
| E      | 4            | 7     | 7      | H         |
| L      | 11           | 14    | 14     | O         |
| L      | 11           | 14    | 14     | O         |
| O      | 14           | 17    | 17     | R         |

*Result: "HELLO" → "KHOOR". Decrypting "KHOOR" with key 3 reverses the process — subtracting 3 from each ciphertext position recovers the original plaintext exactly.*

## ● SECTION 2

## Vigenère Cipher

The Vigenère cipher extends Caesar's idea in one elegant step: instead of a single fixed shift for the entire message, it uses a keyword where each letter defines the shift for the corresponding plaintext letter. The keyword repeats cyclically to cover the full message length, making the cipher polyalphabetic. The practical consequence is immediate — the same plaintext letter encrypts to different ciphertext letters

depending on its position, which breaks the statistical fingerprint that makes Caesar so easy to crack.

### Encryption & Decryption Formulae

**Encrypt:**  $C_i = (P_i + K_i) \bmod 26$

**Decrypt:**  $P_i = (C_i - K_i + 26) \bmod 26$

$K_i$  is the numeric value of the  $i$ -th keyword letter, cycling modulo the keyword length.

### Step-by-Step Example — Encrypting "ATTACK" with keyword "KEY"

The keyword "KEY" has numeric values K=10, E=4, Y=24, then repeats. Each plaintext letter is shifted by its corresponding keyword value:

| Plain | P  | Key | K  | P + K | mod 26 | Cipher |
|-------|----|-----|----|-------|--------|--------|
| A     | 0  | K   | 10 | 10    | 10     | K      |
| T     | 19 | E   | 4  | 23    | 23     | X      |
| T     | 19 | Y   | 24 | 43    | 17     | R      |
| A     | 0  | K   | 10 | 10    | 10     | K      |
| C     | 2  | E   | 4  | 6     | 6      | G      |
| K     | 10 | Y   | 24 | 34    | 8      | I      |

Result: "ATTACK" → "KXRKGI". Notice the two identical T's encrypt to different letters — X and R — because they fall under different keyword letters. This is precisely what defeats naive frequency analysis.

## ● SECTION 3

# Why Caesar Cipher Is Cryptographically Insecure

Despite its historical charm, the Caesar cipher offers essentially no security by modern standards. Three compounding weaknesses make it trivially breakable — and understanding them is a useful lens for appreciating what strong encryption actually requires.

### 1 — A Vanishingly Small Keyspace

The entire Caesar keyspace is 25 keys (shifts 1 through 25; shift 0 is a no-op, and shift 26 wraps back to 0). An attacker can simply try every possible key in under a millisecond on any modern machine. There is no computational barrier to brute force — the search space is smaller than a two-digit number.

### 2 — The Frequency Fingerprint Is Preserved

Because the cipher is monoalphabetic — every plaintext 'E' always maps to the same ciphertext letter — the statistical distribution of English text is preserved in the ciphertext, merely shifted. An attacker who counts letter frequencies and identifies the most common ciphertext letter can determine the key in a single step, without even attempting brute force. In English, 'E' appears roughly 12.7% of the time; if 'H' is the most common ciphertext letter, the key is almost certainly 3.

### 3 — No Diffusion or Confusion

Modern ciphers are built on two core properties: diffusion (changing one input bit should flip roughly half the output bits) and confusion (the relationship between key and ciphertext should be as complex as possible). Caesar has neither — each character is transformed independently, and the structure of the plaintext is structurally mirrored in the ciphertext. Repeating patterns in the plaintext produce repeating patterns in the output. AES and ChaCha20, by contrast, use substitution-permutation networks that ensure avalanche effects across the entire block.

#### ● SECTION 4

## Caesar vs. Vigenère — Fundamental Security Difference

The jump from Caesar to Vigenère is conceptually modest — replace one shift with many — but the security gain is substantial. Caesar is monoalphabetic: it uses a single fixed shift for every character, so identical plaintext letters always produce identical ciphertext letters. Vigenère is polyalphabetic: the shift rotates through a keyword, so the same plaintext letter can produce different ciphertext letters at different positions.

In practical terms, this collapses three critical attack vectors. Brute force becomes infeasible for any reasonably long keyword — a 6-letter keyword yields a keyspace of  $26^6 \approx 300$  million possibilities, compared to 25 for Caesar. Simple frequency analysis on the full ciphertext fails because 'E' no longer has a single fixed ciphertext alias; the distribution appears flattened. And pattern recognition across the message — finding repeated structures in the ciphertext to infer repeated structures in the plaintext — is largely defeated.

That said, Vigenère is not unbreakable. Because the keyword repeats, letters at positions separated by the keyword length are encrypted with the same shift — reverting the problem to an independent Caesar cipher on each positional subset. The Kasiski examination exploits repeated trigrams in the ciphertext to estimate the keyword length, after which each positional group can be broken individually. The Index of Coincidence provides a statistical test to distinguish polyalphabetic ciphertext from random noise. Both techniques were firmly established by the late 19th century, long before the digital age. Today, both ciphers are considered historically interesting but entirely obsolete — modern symmetric encryption (AES-256, ChaCha20) operates on 128-bit blocks with key spaces of  $2^{128}$  to  $2^{256}$ , making brute force computationally impossible.

#### ● SECTION 5

## Frequency Analysis & Cryptanalysis

Frequency analysis is the practice of exploiting the statistical non-uniformity of natural language to break ciphers. The key observation is simple: in English, letters do not appear with equal probability. 'E' occurs roughly 12.7% of the time, followed by T (9.1%), A (8.2%), O (7.5%), N (6.7%), and so on down to Z (0.07%) and Q (0.10%). This distribution is remarkably stable across different English texts, which makes

it a reliable fingerprint for an attacker.

### How It Breaks Caesar

In a Caesar-encrypted message, every 'E' becomes the same ciphertext letter — say 'H' if the key is 3. The entire English frequency distribution is therefore preserved in the ciphertext, just shifted by the key. An attacker who counts all ciphertext letters and identifies the most common one can recover the key in a single arithmetic step:  $\text{key} = (\text{ciphertext\_peak} - \text{'E'}) \bmod 26$ . A chi-squared test formalises this intuition — it scores how closely a candidate decryption's letter distribution matches expected English frequencies, and the candidate with the lowest score is almost certainly correct.

### Chi-Squared Scoring

$$\chi^2 = \sum (\text{observed\_freq} - \text{expected\_freq})^2 / \text{expected\_freq}$$

*A lower  $\chi^2$  value means the distribution is closer to English. All 25 Caesar decryptions are scored and the minimum is returned automatically.*

### Why Vigenère Resists — But Does Not Defeat — The Attack

Polyalphabetic substitution flattens the frequency distribution. Because each plaintext letter is shifted by a different keyword letter, 'E' appears as multiple different ciphertext characters across the message. A naive frequency count on the full ciphertext shows a near-uniform distribution, hiding the English fingerprint entirely. The attack requires a preliminary step: determine the keyword length first (via Kasiski or Index of Coincidence), then split the ciphertext into groups of positions that share a keyword letter, and apply standard frequency analysis independently to each group. Each group reduces to a Caesar problem, and the full keyword is assembled letter by letter. The method works well once sufficient ciphertext is available — generally at least ten times the keyword length — but fails on short messages or one-time-pad variants where the keyword is as long as the message itself.

## ● SECTION 6

# Implementation Summary

The project is implemented in Python 3.9+, structured as a modular package with a Streamlit web dashboard for interactive use. All mandatory and bonus requirements have been implemented and verified.

| Module              | Contents                                            | Status             |
|---------------------|-----------------------------------------------------|--------------------|
| core/caesar.py      | Encrypt, decrypt, brute-force attack                | <b>Mandatory</b> ✓ |
| core/vigenere.py    | Encrypt, decrypt, keyword cycling                   | <b>Mandatory</b> ✓ |
| core/frequency.py   | Letter frequency, chi-squared scoring, key recovery | <b>Bonus</b> ✓     |
| utils/text_utils.py | Input validation, .txt file I/O                     | <b>Bonus</b> ✓     |
| app.py              | Streamlit dashboard — 5 interactive pages           | <b>Extra</b> ✓     |
| tests/              | pytest suite — 75 test cases                        | <b>Extra</b> ✓     |